

When Do Sketches Beat Exact SQL?

Benchmarking Apache DataSketches in Spark SQL

Spenser Sun
Harvard University

Abstract

Distinct-count queries such as “how many unique users appeared in each group and time window” are very common in analytics systems, but exact `COUNT(DISTINCT)` can become expensive because the system must track a lot of unique values across partitions, groups, and time windows. This project evaluates when Apache DataSketches helps in Spark SQL. I built a Scala and Spark benchmark that compares four different types of algorithms: Spark exact distinct counting, Spark `approx_count_distinct`, DataSketches HLL, and Theta sketches. For DataSketches HLL and Theta sketch, I also implement materialized sketch tables to make sketches reusable. I find that Spark exact is a very strong baseline for one-off and warm-cache queries, while materialized HLL works best for repeated approximate grouped and multi-group distinct queries over large cold data. At 100M uniform synthetic rows, the materialized HLL table is about 204x smaller than raw Parquet. For the cold multi-group query, exact Spark takes 1691 ms while querying the HLL sketch table takes 67 ms after a one-time 15540 ms build cost. Theta was also implemented and benchmarked, but HLL is a better fit for this distinct-count workload because it is more compact and usually faster. Theta may be promising for future set-operation workloads.

1 Introduction

Modern analytics systems often ask distinct-count questions: how many unique users visited a service, how many distinct devices appeared in each region, or how many unique customers were active in each time window. In SQL, these become different forms of `COUNT(DISTINCT key)`. Exact distinct counting is useful, but it can be expensive because the system must maintain enough state to identify unique values, often after grouping and shuffling data across a distributed execution engine.

Approximate query processing offers a different tradeoff. If we accept small, measurable error, we may achieve lower

memory, lower storage, or lower query cost. Sketches are a natural fit for approximate distinct counting. A sketch is a compact probabilistic summary of a dataset. It can estimate cardinality, be merged across partitions, and often be stored for later reuse.

This project asks the following question: when do Apache DataSketches help compared with Spark’s native exact and approximate distinct-count operators? The answer is not simply that sketches are faster. Spark’s native exact and approximate operators are highly optimized. DataSketches becomes interesting when sketches are treated as reusable summaries rather than as one-query replacements for `COUNT(DISTINCT)`. The strongest result in this project is materialized HLL. That is, we build compact sketch tables once, store them as Parquet, and answer repeated approximate queries by merging stored sketch bytes instead of rereading the full raw table.

2 Background

2.1 Sketches and Apache DataSketches

A sketch is a compact data structure that summarizes a stream or dataset. For distinct counting, the sketch does not store every unique value. Instead, it stores enough probabilistic state to estimate the number of distinct values with controlled error. Sketches are especially useful in distributed systems because local sketches can be merged into a global sketch.

Apache DataSketches is a library of sketch algorithms rather than a single algorithm. This project focuses on two:

- **HLL**: a HyperLogLog-style sketch specialized for approximate cardinality estimation.
- **Theta**: a more general sketch family that supports cardinality estimation and set operations such as union, intersection, and difference.

Because the benchmark queries in this project are distinct-count queries, HLL is expected to be the better fit. Theta is

still included because it is part of DataSketches and because its broader set-operation support is important future work.

2.2 Spark Baselines

Spark SQL already supports exact distinct counting via `COUNT(DISTINCT)` and approximate distinct counting via `approx_count_distinct`, which uses Spark’s built-in approximate cardinality machinery. These are strong baselines: they are integrated into Spark’s optimizer and execution engine. Therefore, a DataSketches implementation must overcome both algorithmic cost and integration overhead.

3 Goals Achieved

The project achieved its core goal, which is an end-to-end Spark benchmark for studying when Apache DataSketches is useful for distinct-count analytics. In particular, it achieved:

- **Benchmark coverage:** synthetic and NYC Taxi inputs, four distinct-count query shapes, and Spark exact, Spark approximate, DataSketches HLL, and DataSketches Theta baselines.
- **System implementation:** direct DataSketches UDAFs, partition-level sketch aggregation, and reusable sketch materialization for repeated queries.
- **Evaluation coverage:** repeated-trial runtimes, approximation error, sketch-table size, build cost, and break-even query counts across multiple workloads, cache regimes, and data sizes up to 50M rows, plus one 100M stress run.

The main planned goal not fully achieved is evaluating DataSketches on a distributed cluster such as CloudLab. The experiments were run locally on an M1 Pro laptop, which is sufficient for a controlled project prototype but not a full cluster-scale systems evaluation.

4 System Design

4.1 Synthetic Data Model

All synthetic datasets are mapped into the following schema:

- `distinct_key`: the value being counted.
- `group_id`: a grouping dimension.
- `date_bucket`: a time dimension.

This schema supports the four benchmark queries:

- **Global distinct:** count distinct `distinct_key`.

- **Grouped distinct:** count distinct `distinct_key` by `group_id`.
- **Time-window distinct:** count distinct `distinct_key` by `date_bucket`.
- **Multi-group distinct:** count distinct `distinct_key` by `(date_bucket, group_id)`.

4.2 DataSketches Integration

Spark does not provide native Apache DataSketches SQL aggregates. The first implementation integrated HLL and Theta as Spark UDAFs. This was correct, but slow. This is because each row updated a sketch through Spark’s aggregation buffer, which creates high per-row update and serialization/deserialization overhead.

The first optimization was partition-level sketching. Instead of updating serialized sketch buffers row by row, each Spark partition builds mutable sketch objects in memory. Only after local aggregation does the implementation serialize sketch bytes and merge across partitions. This preserves the same approximate result while reducing repeated serialization overhead.

4.3 Sketch Materialization

Partition-level sketching still scans the raw input for each query. The second optimization is sketch materialization. I build sketches once at a chosen logical granularity and store them as a Parquet table:

```
(date_bucket, group_id) -> sketch_bytes
```

Later queries read the sketch table and merge sketch bytes:

- Global distinct merges all stored sketches.
- Grouped distinct merges all dates per group.
- Time-window distinct merges all groups per date.
- Multi-group distinct reads the materialized sketch rows directly.

The materialization granularity can be chosen by the designer. This project uses `(date_bucket, group_id)` because it supports all four query shapes and illustrates how reusable sketches can answer rollups.

5 Benchmark Methodology

5.1 Methods Compared

The benchmark compares:

- Spark exact `COUNT(DISTINCT)`.

- Spark `approx_count_distinct` with relative standard deviation 0.05.
- DataSketches HLL and Theta over raw input using partition-level sketching.
- Materialized HLL and Theta sketch tables.

The original direct UDAF implementation is kept in the codebase for comparison but skipped in later analysis because earlier results showed that it is dominated by partition-level sketching.

5.2 Datasets

The synthetic workloads are:

- **Uniform:** balanced groups and days.
- **High-cardinality:** larger distinct-value space.
- **Many-groups:** 1000 groups, creating many more materialized sketch rows.
- **Skewed:** hot groups and hot keys, approximating uneven real-world distributions.

The size sweep uses 1M, 5M, 10M, 20M, and 50M rows. I also ran an 100M uniform cold-cache experiment. The real-data experiment uses NYC Taxi trip data, mapped so that pickup/dropoff routes form `distinct_key`, pickup location forms `group_id`, and pickup date forms `date_bucket`.

5.3 Warm and Cold Cache Regimes

The benchmark distinguishes warm and cold paths:

- **Warm:** input, sketch table, and measured query results may be cached by Spark.
- **Cold:** Spark-level caching is disabled, so queries read stored Parquet data.

This distinction is important because materialization is intended to reduce raw scan cost. If Spark can cheaply rescan cached raw data, exact SQL is a very strong baseline. Cold-cache experiments test the setting where the compact sketch table has a chance to matter.

5.4 Metrics

The main metrics are:

- **Runtime:** median query time across three measured trials after one warmup.
- **Build time:** median time to build the materialized sketch table.

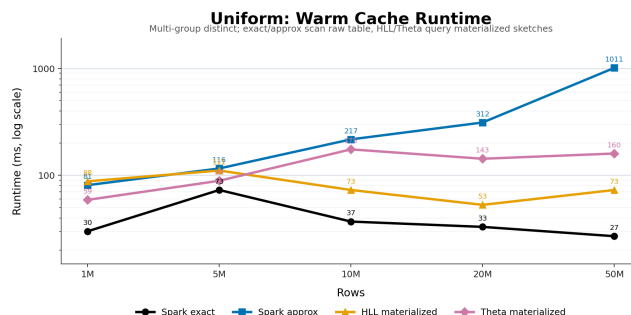


Figure 1: Warm-cache multi-group distinct on uniform synthetic data. Spark exact remains flat and faster because cached raw scans are cheap.

- **Relative error:** $|\text{approx} - \text{exact}| / \text{exact}$.
- **Size ratio:** raw Parquet size divided by materialized sketch-table size.
- **Break-even queries:** build time / (exact query time – sketch query time), when the sketch query is faster than the exact query.

6 Results

6.1 Warm Cache: Spark Exact Is Hard to Beat

Figure 1 shows the warm-cache uniform multi-group query, where the input data is already cheap for Spark to rescan. The black Spark-exact line stays below the materialized-sketch lines at every tested size. At 50M rows, exact Spark takes 27 ms, while materialized HLL takes 73 ms and materialized Theta takes 160 ms. This is an important negative result: materialization is not automatically faster just because sketches are smaller. When the raw table is already cached, Spark’s native execution path is hard to beat.

6.2 Cold Cache: HLL Materialization Helps

Figure 2 shows the same uniform multi-group query under the cold-cache regime. Here Spark must read stored Parquet data, so the compact materialized HLL table has a clear advantage. At 50M rows, exact Spark takes 719 ms and Spark approximate also takes 719 ms, while materialized HLL takes 70 ms. The figure also shows a scaling difference: exact and Spark approximate grow with the raw input scan, but the HLL sketch-query line remains comparatively flat because the query reads the much smaller sketch table. In the 100M uniform cold run, exact Spark takes 1691 ms, while materialized HLL takes 67 ms after the one-time sketch build.

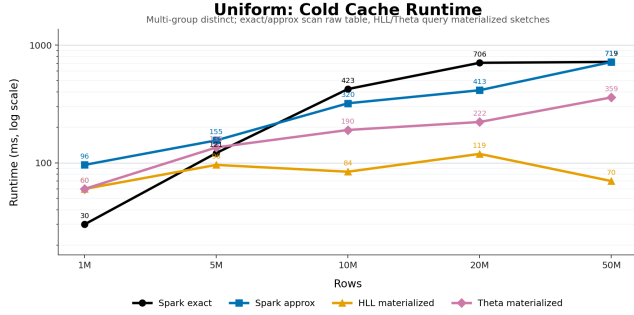


Figure 2: Cold-cache multi-group distinct on uniform synthetic data. Materialized HLL remains low as raw-table scan cost grows.

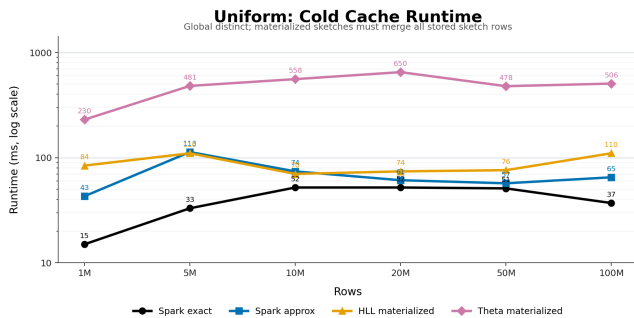


Figure 3: Cold-cache global distinct on uniform synthetic data. Because global distinct merges all materialized sketches, materialization is less compelling.

6.3 Query Shape Matters

Figure 3 shows why materialization does not help all query shapes equally. The sketch table is materialized at `(date_bucket, group_id)`, so a multi-group query aligns with the stored granularity, but a global distinct query must merge every stored sketch. In this global query, exact Spark remains fast even in the cold-cache setting. At 100M rows, exact Spark takes 37 ms, Spark approximate takes 65 ms, materialized HLL takes 110 ms, and materialized Theta takes 506 ms. This explains why the best use case is repeated grouped or multi-group analytics, not every distinct-count query.

6.4 Workload Shape Matters

Table 1 summarizes the 50M cold-cache HLL materialization results across workload shapes. The main pattern is that materialized HLL is consistently useful for the multi-group query. The speedup is largest when the sketch table is compact relative to raw data, for example uniform (719 ms exact versus 70 ms HLL) and skewed (446 ms exact versus 73 ms HLL). The exception is the many-groups grouped query, where HLL takes 437 ms and exact Spark takes 400 ms. That workload creates many more materialized sketch rows, reducing the

storage advantage and increasing sketch-query overhead.

6.5 Size and Accuracy Tradeoff

Table 2 explains the mechanism behind the runtime results and the price paid for it. HLL materialization greatly reduces the amount of data read at query time: for 50M cold multi-group experiments, the HLL table is 100.7x smaller than raw Parquet on uniform data, 108.3x smaller on high-cardinality data, and 149.1x smaller on skewed data. Theta is much less compact in the same workload, ranging from 1.6x to 16.9x. The many-groups workload is the weakest case for both sketches because it creates many more `(date_bucket, group_id)` sketch rows and reduces the compression ratio.

The same table shows that HLL median relative error stays close to or below 1%, with p95 error below 3% in all four workloads. Theta sometimes has lower median error, including zero median error for many-groups and skewed, but this comes with much larger sketch tables. For this project’s pure distinct-count workload, the better engineering tradeoff is therefore HLL: it gives large storage reduction and fast materialized queries while keeping error small enough for approximate analytics.

7 Discussion

The results indicate that sketch performance depends as much on physical design as on the sketch algorithm itself. When sketches are evaluated as one-query aggregates over raw input, they still pay the cost of scanning the full dataset and integrating with Spark’s aggregation machinery. Their advantage becomes clearer when sketches are materialized as reusable summaries and the workload contains repeated queries over compatible grouping dimensions.

The warm and cold cache results also show that approximate query processing should be evaluated under explicit assumptions about data access cost. A cached raw table favors Spark’s native operators, while a cold or repeatedly accessed table gives compact sketch summaries more opportunity to reduce I/O and query latency. Therefore, sketch materialization is best understood as a workload-level optimization rather than a drop-in replacement for exact distinct counting.

Finally, HLL and Theta have different roles. HLL specializes in approximate distinct counting and is much more compact in this workload. Theta is more general and should be evaluated on richer set operations before concluding that it is a poor sketch in general.

8 Limitations

This project has several limitations. Firstly, the experiments ran on a local M1 Pro laptop rather than a distributed Spark cluster. Also, the Spark caching and local OS cache behavior

Workload	Query	ExactQ	HLLQ	Build	Ratio	BE
Uniform	Grouped	1298	93	7483	100.7	6.21
Uniform	Multi	719	70	7483	100.7	11.53
High-card.	Grouped	418	85	6348	108.3	19.06
High-card.	Multi	663	72	6348	108.3	10.74
Many-groups	Grouped	400	437	11914	13.7	never
Many-groups	Multi	1049	202	11914	13.7	14.07
Skewed	Grouped	393	80	5093	149.1	16.27
Skewed	Multi	446	73	5093	149.1	13.65

Table 1: 50M cold-cache materialized HLL results. Times are milliseconds. Ratio is raw Parquet size divided by HLL sketch-table size. BE is the number of repeated queries needed to amortize build cost.

Workload	HLL Size	Theta Size	HLL Med.	HLL P95	Theta Med.	Theta P95
Uniform	100.7x	6.5x	0.98%	2.81%	0.91%	2.61%
High-card.	108.3x	6.9x	0.94%	2.73%	0.90%	2.64%
Many-groups	13.7x	1.6x	0.61%	1.77%	0.00%	0.00%
Skewed	149.1x	16.9x	0.60%	1.91%	0.00%	1.53%

Table 2: Storage and accuracy tradeoff for 50M cold multi-group distinct experiments. Size columns report raw Parquet size divided by materialized sketch-table size. Error columns report relative error.

can affect measured runtime. For the workload, my synthetic workloads isolate mechanisms but simplify real analytics data. The benchmark also focuses on distinct counting and it does not exercise Theta’s strongest set-operation use cases.

9 Future Work

Future work should run the benchmark on a distributed cluster such as CloudLab, where shuffle, spill, executor memory, and distributed storage effects can be measured directly. The workload should also include richer analytics benchmarks such as TPC-H, TPC-DS, or Star Schema Benchmark style distinct-count queries with joins and filters. Materialization granularity should be explored systematically: `group_id`, `date_bucket`, `(date_bucket, group_id)`, and higher-dimensional keys trade storage size for query flexibility. Finally, Theta should be benchmarked on set-operation workloads such as union, intersection, difference, retention, and cohort overlap, where it is expected to be more valuable than in pure distinct counting.

10 AI Usage Report

I used AI assistance mainly as a person helping to brainstorm and a programming partner. The AI helped me:

- **Brainstorm preliminary ideas.** Early in the project, I used AI to compare possible directions, such as whether to focus on exact Spark baselines, Spark’s built-in approximation, direct DataSketches UDAFs, or sketch ma-

terialization. This helped me narrow the project into a benchmark with a clear systems question.

- **Make the code cleaner and more modular.** I used AI to help organize the benchmark runner, configuration parsing, query definitions, metrics code, synthetic data generation, and materialization code into separate modules. I still checked the design myself, but AI was useful for keeping repetitive Scala/Spark code consistent.
- **Debug implementation issues.** AI helped with several Spark and DataSketches integration problems. One important example was the slow direct UDAF path: instead of only treating it as a bug, the debugging process helped identify UDAF serialization and aggregation overhead as a real systems issue, which motivated partition-level sketching and later sketch materialization.
- **Improve the presentation and writing.** For the presentation, AI gave useful suggestions on how to structure the story: motivation, what sketches are, why direct UDAFs were not enough, why materialization matters, and how to explain warm-cache versus cold-cache results. It also helped turn benchmark tables and plots into clearer claims.

What I learned from using AI is that it is most useful when I ask it to compare design alternatives and explain tradeoffs, not merely to write code. In this project, the best AI interactions were the ones where I gave concrete constraints, asked for competing explanations, and then checked the suggestions against benchmark results.

What surprised me is that approximate sketches did not automatically beat Spark exact. Spark’s native operators and caching were much stronger than I initially expected. The AI was useful in pushing me to treat this as a systems result rather than a failure: sketches help under specific physical-design conditions, especially repeated cold queries over compact materialized summaries.

I also have some useful tips from this experience:

- AI is especially helpful for tasks that are conceptually easy but require using unfamiliar or verbose libraries, such as organizing benchmark CSV outputs, aggregating results, and generating plots.
- For larger code changes, it works better to write the skeleton or interface myself first and then ask AI to fill in details. This keeps the design under my control while still using AI for implementation speed.
- Specific prompts matter. AI does not work as well when asked vaguely to “implement this feature.” It works much

better when the inputs, expected outputs, constraints, and edge cases are stated clearly.

- AI-generated code still need verification. I found it important to inspect benchmark numbers, rerun experiments when needed, and make sure the final claims matched the actual data.

11 Conclusion

This project benchmarked Apache DataSketches in Spark SQL and found that sketches are most useful as reusable approximate summaries, not as universal replacements for exact SQL. Spark exact remains a strong baseline for warm-cache and one-off queries. Materialized HLL provides the clearest win for repeated cold-cache grouped and multi-group distinct-count workloads, where the sketch table can be 100x to 200x smaller than raw Parquet while maintaining around 1% median error in large runs. Theta is implemented and benchmarked, but its main value likely lies in future set-operation queries rather than this project’s pure distinct-count workload.